

SIMATS ENGINEERING
ASSESSMENT TOOL 1: Knowledge Representation Design
Assessment

COURSE NAME: **ARTIFICIAL INTELLIGENCE COURSE CODE: MLA01**
AND EXPERT SYSTEMS

COURSE OUTCOME COVERED

CO3: *Implement propositional and first-order logic using resolution, unification, and inference techniques for knowledge representation and reasoning.(BTL 3)*

Assessment Tool Weightage: 40 %

Total Marks: 100

Time: 60 Mins

No. of Questions: 5

Annexure A
Knowledge Representation Design Assessment

Code of Conduct:

I _S.Venkateswarlu /192425340__ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: S.Venkateswarlu

Criterion	Weightage	Marks Obtained
1. Problem Analysis and Domain Understanding	10%	
2. Knowledge Engineering and Knowledge Base Design	15%	
3. Propositional Logic Representation	10%	
4. First-Order Logic Representation	15%	
5. Unification and Resolution	15%	
6. Forward and Backward Chaining	15%	
7. Inference and Reasoning Accuracy	10%	

8. System Architecture / Representation Diagram	5%	
9. Prototype Implementation and Results	5%	
Total	100%	

Signature of the Faculty

Total Marks Awarded

ANSWER ANY ONE QUESTION

1. Smart Healthcare Diagnosis System

A hospital wants to develop an AI-based system that can identify possible illnesses from patient symptoms. The system contains facts such as:

- Ravi has fever.
- Ravi has cough.
- Ravi has body pain.
- Patients with fever and cough may have flu.
- Patients with flu and body pain may have viral infection.
- Patients with viral infection should be advised to rest.

Task:

1. Represent the given knowledge using **Propositional Logic**.
2. Convert the knowledge into **First-Order Logic (FOL)**.
3. Construct a knowledge base containing facts and rules.
4. Demonstrate **unification** for at least two predicates.
5. Use **forward chaining** to determine Ravi's condition.
6. Use **backward chaining** to prove whether Ravi needs rest.
7. Demonstrate **resolution** for one selected query.
8. Explain the final inference produced by the system.

ANSWER:

Smart Healthcare Diagnosis System

1. Problem Analysis and Domain Understanding

The proposed Smart Healthcare Diagnosis System is a rule-based AI system that identifies possible illnesses from patient symptoms.

Objective

The objective is to use facts about a patient and medical rules to infer the patient's possible condition and recommend appropriate action.

Given Patient

Patient: Ravi

Known symptoms:

- Ravi has fever.
- Ravi has cough.
- Ravi has body pain.

Medical Rules

1. If a patient has fever and cough, the patient may have flu.
2. If a patient has flu and body pain, the patient may have viral infection.
3. If a patient has viral infection, the patient should be advised to rest.

Entities

- Patient: Ravi
- Diseases/conditions: Flu, Viral Infection
- Symptoms: Fever, Cough, Body Pain
- Recommendation: Rest

2. Knowledge Engineering and Knowledge Base Design

The knowledge base consists of facts and rules.

Facts

Let:

- Fever(Ravi)

- Cough(Ravi)
- BodyPain(Ravi)

These represent the symptoms observed in Ravi.

Rules

Rule 1:

If a patient has fever and cough, then the patient may have flu.

[
 $\text{Fever}(x) \wedge \text{Cough}(x) \rightarrow \text{Flu}(x)$
]

Rule 2:

If a patient has flu and body pain, then the patient may have viral infection.

[
 $\text{Flu}(x) \wedge \text{BodyPain}(x) \rightarrow \text{ViralInfection}(x)$
]

Rule 3:

If a patient has viral infection, then the patient should rest.

[
 $\text{ViralInfection}(x) \rightarrow \text{NeedsRest}(x)$
]

Knowledge Base

[
 $\text{KB} = \{\text{Fever}(\text{Ravi}), \text{Cough}(\text{Ravi}), \text{BodyPain}(\text{Ravi}),$
 $\text{R1}, \text{R2}, \text{R3}\}$
]

where:

[
 $\text{R1: Fever}(x) \wedge \text{Cough}(x) \rightarrow \text{Flu}(x)$
]

[
 $\text{R2: Flu}(x) \wedge \text{BodyPain}(x) \rightarrow \text{ViralInfection}(x)$
]

[
R3: $\text{ViralInfection}(x) \rightarrow \text{NeedsRest}(x)$
]

3. Propositional Logic Representation

In propositional logic, each statement is represented by a proposition.

Let:

- F = Ravi has fever.
- C = Ravi has cough.
- B = Ravi has body pain.
- $F1$ = Ravi has flu.
- V = Ravi has viral infection.
- R = Ravi needs rest.

Facts

[
 $F = \text{True}$
]

[
 $C = \text{True}$
]

[
 $B = \text{True}$
]

Rules

Rule 1:

[
 $(F \wedge C) \rightarrow F1$
]

Rule 2:

[
 $(F1 \wedge B) \rightarrow V$
]

Rule 3:

[
 $V \rightarrow R$
]

Complete Propositional Knowledge

[
 $F \wedge C \wedge B$
]

[
 $(F \wedge C) \rightarrow F_I$
]

[
 $(F_I \wedge B) \rightarrow V$
]

[
 $V \rightarrow R$
]

Therefore:

[
 $F, C, B \rightarrow F_I \rightarrow V \rightarrow R$
]

Hence, Ravi needs rest.

4. First-Order Logic Representation

First-Order Logic allows us to represent general rules using variables.

Facts

[
 $\text{Fever}(\text{Ravi})$
]

[
 $\text{Cough}(\text{Ravi})$
]

[
BodyPain(Ravi)
]

Rule 1

For every patient, if the patient has fever and cough, then the patient has flu.

[
 $\forall x[(\text{Fever}(x) \wedge \text{Cough}(x)) \rightarrow \text{Flu}(x)]$
]

Rule 2

For every patient, if the patient has flu and body pain, then the patient has viral infection.

[
 $\forall x[(\text{Flu}(x) \wedge \text{BodyPain}(x)) \rightarrow \text{ViralInfection}(x)]$
]

Rule 3

For every patient, if the patient has viral infection, then the patient needs rest.

[
 $\forall x[\text{ViralInfection}(x) \rightarrow \text{NeedsRest}(x)]$
]

FOL Knowledge Base

[
KB =
{
Fever(Ravi),
Cough(Ravi),
BodyPain(Ravi),
R1,R2,R3
}
]

This representation allows the same rules to be applied to any patient.

5. Unification and Substitution

Unification is the process of finding substitutions that make two predicates identical.

Unification Example 1

Consider:

[
Fever(x)
]

and

[
Fever(Ravi)
]

The substitution is:

[
{x/Ravi}
]

After substitution:

[
Fever(Ravi)
]

Therefore, the two predicates successfully unify.

Unification Example 2

Consider:

[
NeedsRest(x)
]

and

[
NeedsRest(Ravi)
]

The substitution is:

[
{x/Ravi}
]

After substitution:

[
NeedsRest(Ravi)
]

Therefore, they successfully unify.

Most General Unifier

For both examples:

[
MGU = {x/Ravi}
]

6. Forward Chaining

Forward chaining is a data-driven reasoning method. It starts with known facts and repeatedly applies rules to derive new facts.

Initial Facts

[
Fever(Ravi)
]
[
Cough(Ravi)
]
[
BodyPain(Ravi)
]

Step 1: Apply Rule 1

Rule:

[
Fever(x) ∧ Cough(x) → Flu(x)
]

Substitute:

[
x=Ravi
]

We have:

[
Fever(Ravi)
]

and

[
Cough(Ravi)
]

Therefore:

[
Flu(Ravi)
]

Step 2: Apply Rule 2

Rule:

[
 $\text{Flu}(x) \wedge \text{BodyPain}(x) \rightarrow \text{ViralInfection}(x)$
]

We have:

[
Flu(Ravi)
]

and:

[
BodyPain(Ravi)
]

Therefore:

[
ViralInfection(Ravi)
]

Step 3: Apply Rule 3

Rule:

[
ViralInfection(x) $\rightarrow$ NeedsRest(x)
]

We have:

[
ViralInfection(Ravi)
]

Therefore:

[
NeedsRest(Ravi)
]

Forward Chaining Result

[
Fever(Ravi)
]

↓

[
Flu(Ravi)
]

↓

[
ViralInfection(Ravi)
]

↓

[
NeedsRest(Ravi)
]

Therefore, Ravi should be advised to rest.

7. Backward Chaining

Backward chaining is a goal-driven reasoning method. It starts with a goal and works backward to determine whether the required facts are available.

Query

[
NeedsRest(Ravi)
]

We want to prove:

[
NeedsRest(Ravi)
]

Step 1

Find a rule whose conclusion is NeedsRest.

Rule 3:

[
 $\text{ViralInfection}(x) \rightarrow \text{NeedsRest}(x)$
]

For Ravi:

[
 $\text{ViralInfection}(\text{Ravi}) \rightarrow \text{NeedsRest}(\text{Ravi})$
]

Therefore, we need to prove:

[
 $\text{ViralInfection}(\text{Ravi})$
]

Step 2

Find a rule that concludes ViralInfection.

Rule 2:

[
 $\text{Flu}(x) \wedge \text{BodyPain}(x) \rightarrow \text{ViralInfection}(x)$
]

For Ravi:

[
 $\text{Flu}(\text{Ravi}) \wedge \text{BodyPain}(\text{Ravi})$
]

We already know:

[
BodyPain(Ravi)
]

Now we need:

[
Flu(Ravi)
]

Step 3

Find a rule that concludes Flu.

Rule 1:

[
Fever(x)\land Cough(x)\rightarrow Flu(x)
]

For Ravi:

[
Fever(Ravi)\land Cough(Ravi)
]

Both facts are available.

Therefore:

[
Flu(Ravi)
]

Then:

[
ViralInfection(Ravi)
]

Then:

[
NeedsRest(Ravi)
]

Backward Chaining Conclusion

The query is successfully proved:

$$\boxed{\text{NeedsRest(Ravi)}}$$

8. Resolution

Resolution is an inference technique used to prove whether a query logically follows from the knowledge base.

We will prove:

$$\text{NeedsRest(Ravi)}$$

Convert Rule 3 into Clause Form

Original rule:

$$\text{ViralInfection}(x) \rightarrow \text{NeedsRest}(x)$$

Using:

$$P \rightarrow Q \equiv \neg P \vee Q$$

we get:

$$\neg \text{ViralInfection}(x) \vee \text{NeedsRest}(x)$$

For Ravi:

$$\neg \text{ViralInfection(Ravi)} \vee \text{NeedsRest(Ravi)}$$

Rule 2

$$\text{Flu}(x) \wedge \text{BodyPain}(x) \rightarrow \text{ViralInfection}(x)$$

Clause form:

[
 $\neg \text{Flu}(x) \vee \neg \text{BodyPain}(x) \vee \text{ViralInfection}(x)$
]

For Ravi:

[
 $\neg \text{Flu}(\text{Ravi}) \vee \neg \text{BodyPain}(\text{Ravi}) \vee \text{ViralInfection}(\text{Ravi})$
]

Rule 1

[
 $\text{Fever}(x) \wedge \text{Cough}(x) \rightarrow \text{Flu}(x)$
]

Clause form:

[
 $\neg \text{Fever}(x) \vee \neg \text{Cough}(x) \vee \text{Flu}(x)$
]

For Ravi:

[
 $\neg \text{Fever}(\text{Ravi}) \vee \neg \text{Cough}(\text{Ravi}) \vee \text{Flu}(\text{Ravi})$
]

Facts

[
 $\text{Fever}(\text{Ravi})$
]

[
 $\text{Cough}(\text{Ravi})$
]

[
 $\text{BodyPain}(\text{Ravi})$
]

Negated Query

To prove:

[
NeedsRest(Ravi)
]

assume the opposite:

[
\neg NeedsRest(Ravi)
]

Resolution Step 1

From:

[
\neg Fever(Ravi)\lor\neg Cough(Ravi)\lor Flu(Ravi)
]

and:

[
Fever(Ravi)
]

we derive:

[
\neg Cough(Ravi)\lor Flu(Ravi)
]

Using:

[
Cough(Ravi)
]

we derive:

[
Flu(Ravi)
]

Resolution Step 2

From:

[
¬Flu(Ravi)∨¬BodyPain(Ravi)∨ViralInfection(Ravi)
]

and:

[
Flu(Ravi)
]

we get:

[
¬BodyPain(Ravi)∨ViralInfection(Ravi)
]

Using:

[
BodyPain(Ravi)
]

we derive:

[
ViralInfection(Ravi)
]

Resolution Step 3

From:

[
¬ViralInfection(Ravi)∨NeedsRest(Ravi)
]

and:

[
ViralInfection(Ravi)
]

we derive:

[
NeedsRest(Ravi)
]

But we initially assumed:

[
 $\neg \text{NeedsRest}(\text{Ravi})$
]

Therefore, we obtain a contradiction:

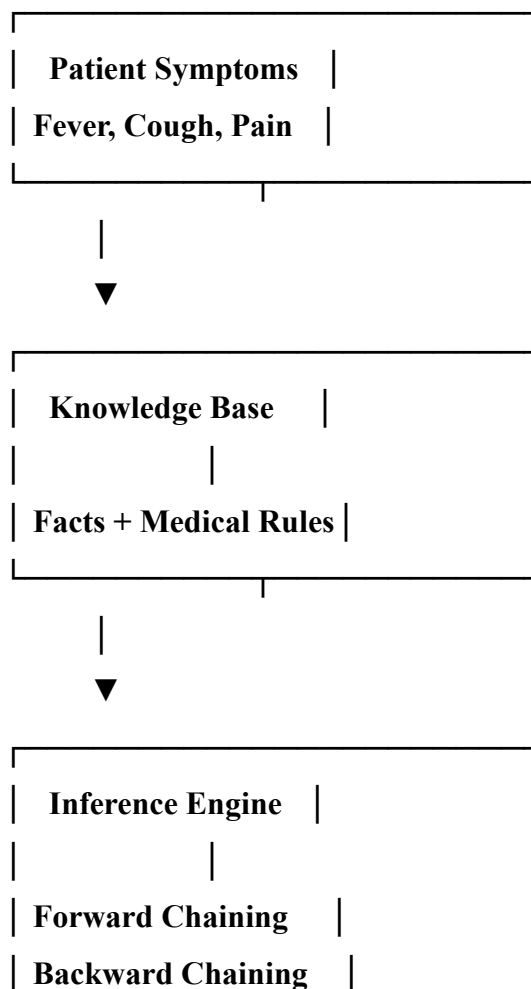
[
 $\text{NeedsRest}(\text{Ravi}) \wedge \neg \text{NeedsRest}(\text{Ravi})$
]

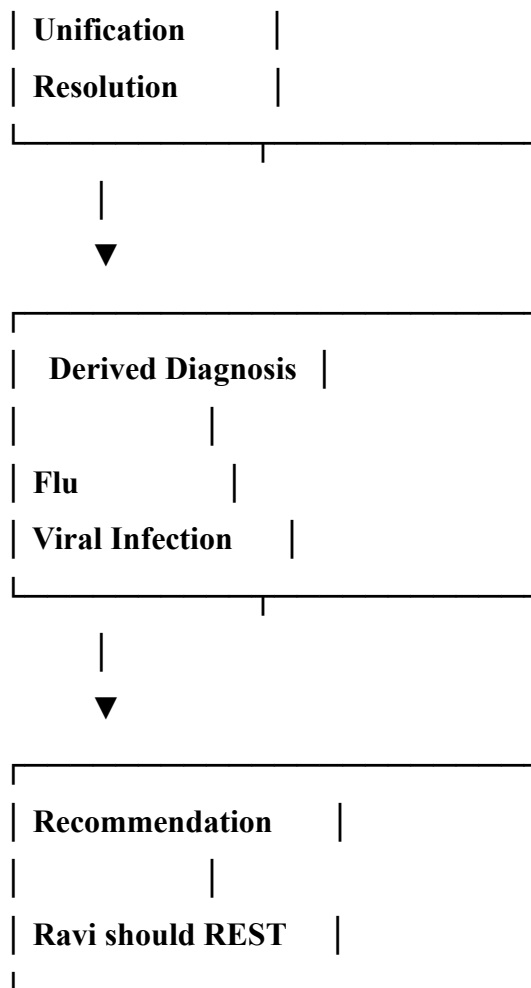
Hence, the negated query is false and the original query is true.

[
 $\boxed{\text{NeedsRest}(\text{Ravi})}$
]

9. System Architecture / Representation Diagram

The architecture of the Smart Healthcare Diagnosis System can be represented as:





10. Complete Inference Trace

The complete reasoning process is:

Initial Facts

[
Fever(Ravi)
]

[
Cough(Ravi)
]

[
BodyPain(Ravi)
]

Inference 1

Fever + Cough

```
[
\Rightarrow Flu(Ravi)
]
Inference 2
Flu + Body Pain
[
\Rightarrow ViralInfection(Ravi)
]
```

```
Inference 3
Viral Infection
[
\Rightarrow NeedsRest(Ravi)
]
Therefore:
[
\boxed{Ravi\ has\ a\ possible\ viral\ infection\ and\ should\ be\ advised\ to\ rest.}
]
```

11. Final Answer / Reasoning Accuracy

The Smart Healthcare Diagnosis System successfully derives the following conclusions:

Stage	Knowledge
Initial fact	Ravi has fever
Initial fact	Ravi has cough
Initial fact	Ravi has body pain
Rule 1	Ravi may have flu
Rule 2	Ravi may have viral infection
Rule 3	Ravi needs rest

```
Final Inference
[
Fever(Ravi)+Cough(Ravi)
```

$$\begin{aligned}
& \rightarrow \text{Flu(Ravi)} \\
&] \\
& [\\
& \text{Flu(Ravi)} + \text{BodyPain(Ravi)} \\
& \rightarrow \text{ViralInfection(Ravi)} \\
&] \\
& [\\
& \text{ViralInfection(Ravi)} \\
& \rightarrow \text{NeedsRest(Ravi)} \\
&]
\end{aligned}$$

Thus, the system concludes that Ravi may have a viral infection and should be advised to rest. The system does not establish a medically confirmed diagnosis; it demonstrates logical inference from the supplied knowledge base.

12. Conclusion

The Smart Healthcare Diagnosis System demonstrates how knowledge representation and logical reasoning can be used in an AI-based expert system. Propositional Logic represents simple statements, while First-Order Logic represents general medical rules using variables and predicates. Unification enables predicates to match with appropriate substitutions. Forward chaining derives conclusions from available symptoms, while backward chaining starts from a desired conclusion and verifies the supporting facts. Resolution provides a formal method for proving the selected query.

The final inference is:

$$\begin{aligned}
& [\\
& \boxed{\text{NeedsRest(Ravi)} = \text{True}} \\
&]
\end{aligned}$$

Therefore, according to the given knowledge base, Ravi should be advised to rest.

